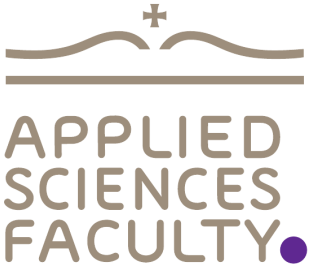
C++ — Додаток. CMake





©Олег Фаренюк,
Creative Commons - Attribution Share Alike (CC BY-SA)
https://www.facebook.com/oleg.farenyuk
indrekis@gmail.com

Огляд CMake

- CMake – кросплатформова система генерації сценаріїв для автоматичної побудови програм.
  - На загал – більш зручна ніж класичний GNU Autotools.
- Може генерувати сценарії для різних варіантів Make[1] та Ninja, проекти середовищ розробки (IDE), Visual Studio, Eclipse, Code::Blocks, CodeLite, Kate, XCode, Sublime тощо.
  - Ремарка: набір генераторів залежить від того, як компілювався CMake – набір буде різним на різних системах.
- Типове використання двостадійне – спершу генерується проект під бажане середовище розробки чи систему побудови, за допомогою якої здійснюється вже безпосередньо компіляція.
  - Другий етап теж можна виконати викликом CMake.

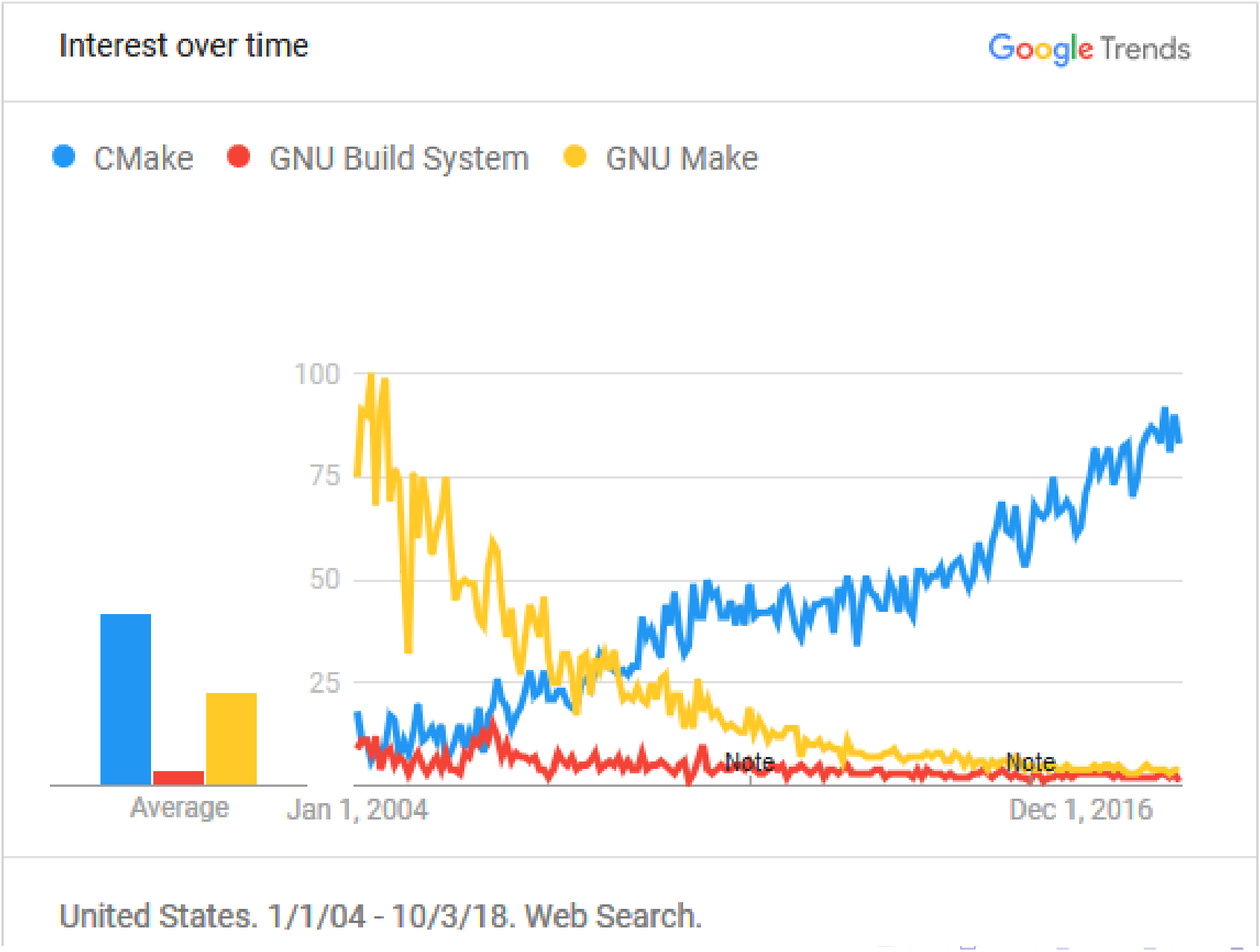
[1] Unix/GNU, MS nmake, MINGW та MSYS тощо.

Головні можливості

- Підтримує як **Out-Of-Source** Builds, так і категорично не рекомендовані[2] In-Source Builds.
- Може створювати як виконавчі програми, так і статичні та динамічні бібліотеки.
- Вміє генерувати код для подальшої побудови, інструментами, які будуються (компілюються) в процесі.
- Дозволяє здійснювати пошук бібліотек та інструментів компіляції, як обов'язкових для побудови програм, так і опціональних.
- Підтримує крос-компіляцію – компіляцію на одній платформі (CPU + OS + компілятор) для іншої.

[2] А тому – заборонені у нашому курсі.

Щодо популярності:



Дуже простий приклад

- Розглянемо приклад програми, який використовувався і для ілюстрації використання make.
  - Вона складається з 5 файлів: *file1.cpp*, *file2.cpp*, *main_file.cpp*, *file1.h*, *file2.h*.
- In-Source build – зло, тому компілюватимемо в окремій директорії.
- **Файл повинен називатися CMakeLists.txt**. Бажано, щоб він знаходився у тій же директорії, що і тексти програм (щодо піддиректорій – див. далі).
- (Майже) найпростіший такий файл:

```
CMAKE_MINIMUM_REQUIRED(VERSION 3.8)

PROJECT(sample_project)

ADD_EXECUTABLE(hello1 file1.cpp file2.cpp
                      main_file.cpp file1.h file2.h)
```

- PROJECT – ім'я проекту (необов'язково, але бажано).
- ADD_EXECUTABLE – вказує створити програму із іменем hello1 із заданих файлів. Для Windows – додасть розширення exe.

Приклад використання – 1

```
$ ls
CMakeLists.txt  file1.cpp  file1.h  file2.cpp  file2.h  main_file.cpp
$ mkdir build
$ cd build
$ cmake -G"Unix Makefiles" ..
-- The C compiler identification is GNU 8.2.0
-- The CXX compiler identification is GNU 8.2.0
-- Check for working C compiler: D:/WorkComp/msys2_64_2015/mingw64/bin/cc.exe
-- Check for working C compiler: D:/WorkComp/msys2_64_2015/mingw64/bin/cc.exe -- works
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Detecting C compile features
-- Detecting C compile features - done
-- Check for working CXX compiler: D:/WorkComp/msys2_64_2015/mingw64/bin/c++.exe
-- Check for working CXX compiler: D:/WorkComp/msys2_64_2015/mingw64/bin/c++.exe -- wo
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - done
-- Detecting CXX compile features
-- Detecting CXX compile features - done
-- Configuring done
-- Generating done
-- Build files have been written to: D:/WorkTeaching/Cpp/lectures/Projects/cmake_demo/
$ make
Scanning dependencies of target hello1
[ 25%] Building CXX object CMakeFiles/hello1.dir/file1.cpp.obj
```

Аналіз прикладу – 1

```
$ mkdir build && cd build
$ cmake -G"Unix Makefiles" ..
-- The C compiler identification is GNU 8.2.0
<.................. skipped ..................>
-- Build files have been written to: <...>/cmake_demo/build
$ make
```

- В окремій (out of source) директорії, викликаємо CMake, вказуємо йому створити генератор "Unix Makefiles", користуючись для CMakeLists.txt, що знаходиться кроком вище (`..`). Директорія ця може бути де завгодно – не обов'язково піддиректорією.
- Якщо CMake повідомив "Generating done", можна компілювати. У цьому випадку – викликом make.
- Не залежно від використаного типу проекту, скомпілювати отриманий таким чином проект можна так:

  ```
  $ mkdir build && cd build
  $ cmake -G"Visual Studio 16 2019" ..
  <.................. skipped ..................>
  $ cmake --build .
  ```

  - Ціль можна вказати за допомогою `--target name`, опції для системи побудови передаються після `--`.
  - Це позбавляє необхідності запускати середовище або згадувати хитрі команди, щоб лише скомпілювати.

Target – цілі компіляції

- Ключовим поняттям CMake, як і Make, є target.
- Наступна команда додає ціль – виконавчу програму hello1:

```
ADD_EXECUTABLE(hello1 file1.cpp file2.cpp
                      main_file.cpp file1.h file2.h)
```

- Існують цілі для створення бібліотек, інсталяції, тестування – з про них далі.
- В одному проекті може бути багато цілей, в тому числі – взаємно пов'язаних.
- ◇ Можна імпортувати цілі, що вже існують – створювати ціль, видиму CMake, яка описує файли – бібліотеки, виконавчі файли тощо, створені незалежно від цього CMake-скрипта.
- ◇ Також, можна викликати користувацькі команди, щоб виконувати операції побудови, невідомі CMake[3].
- Цілі та деякі інші сутності використовують пов'язані із ними властивості – PROPERTY, приклади див. далі.

[3] Однак, такі дії варто мінімізувати – використовувати лише за відсутності інших, більш високорівневих, варіантів.

Мови проекту

- PROJECT може вказувати використані проектом мови програмування:

```
PROJECT(sample_project CXX)
```

- За замовчуванням, вважається, що це C та CXX[4].
- ◇ Інші варіанти – CUDA, Fortran, ASM[5].
- ◇ Є мінімальна підтримка інших мов – див. документацію.
- Це евристика, але, зазвичай, *.c вважатимуться файлами C а *.cpp, *.cxx, *.C – файлами C++.
- Забігаючи наперед, для конкретного файлу мову можна вказати так:

```
set_source_files_properties(${SomeSources} PROPERTIES
                                           LANGUAGE CXX)
```

[4] Так позначається C++.

[5] Варто вказувати останнім у списку – драйвери компіляторів часто вміють із ним працювати безпосередньо.

Діалекти мов програмування

- Є можливість вказувати, якому стандарту повинен відповідати компілятор:

```
set(CMAKE_C_STANDARD 99)
set(CMAKE_C_EXTENSIONS FALSE)
set(CMAKE_CXX_STANDARD 14)
set(CMAKE_CXX_EXTENSIONS TRUE) # Не те, щоб це хороша ідея була
```

- Підтримувані стандарти – див. `https://cmake.org/cmake/help/latest/prop_tgt/CXX_STANDARD.html`.
- Важливо, що таким способом буде вибрано опцію саме того компілятора, для якого створюється сценарій побудови (компіляції) – для GCC буде використано -std=c++14, для MSVC – /std:c++14 і т.д.
- Можна також керувати стандартом для окремих цілей:

```
set_property(TARGET target_name PROPERTY CXX_STANDARD 17)
```

- Можна також вказувати засоби мови, які потрібні, а CMake підбере відповідні ключі:

```
target_compile_features(mylib PRIVATE cxx_constexpr)
```

- ◇ Повний список таких засобів, відомий CMake: `https://cmake.org/cmake/help/latest/prop_gbl/CMAKE_CXX_KNOWN_FEATURES.html`, але **зважайте на версію CMake**, що використовуєте.

Мова CMake

- Робота CMake користується своєрідною скриптовою мовою, із змінними, контрольними інструкціями і т.д.
- Мова ця, в основному, декларативна. Місцями – дещо дивна.
- Варто описувати компіляцію вашого проекту максимально декларативно і завжди максимально уникати втручання в процес компіляції – надаючи перевагу високорівневим засобам.
  - Один приклад був вище – краще писати `set(CMAKE_CXX_STANDARD 14)`, ніж вручну додавати прапорець для конкретного компілятора, `-std=c++14`, `/std:c++14`, намарно ламаючи сумісність із іншими компіляторами.
  - Подробиці – і ще такі ситуації, і як втрутитися, якщо таки немає інших способів – нижче.

Змінні та інші найпростіші конструкції

- Список файлів краще виносити в окрему **змінну**. Зробити це можна так:

```
SET(hello1_FILES file1.cpp file2.cpp
            main_file.cpp file1.h file2.h)
ADD_EXECUTABLE(hello1  ${hello1_FILES})
```

- Символ # – коментар, як в багатьох інших скриптових мовах:

```
ADD_EXECUTABLE(hello1_test  ${hello1_test_FILES}) # Additiona tests
```

- MESSAGE() виводить повідомлення, за потреби може звертатися до змінних середовища чи припиняти компіляцію (FATAL_ERROR):

```
message("CXX Standard is:" ${CMAKE_CXX_STANDARD})
message(FATAL_ERROR "Wrong libxyz version.")
message(STATUS $ENV{PATH})
```

  - ◇ Текст без лапок буде об'єднано – пробіли ігноруватимуться.
  - ◇ Можливі значення першого аргументу: STATUS, WARNING, AUTHOR_WARNING, SEND_ERROR, FATAL_ERROR, DEPRECATION.

Чутливість до регістру

- Чутливість до регістру літер у CMake дещо дивна чи й непослідовна.
- Команди, типу ADD_EXECUTABLE чи SET до регістру не чутливі.
- Однак, змінні – чутливі, для них великі і малі літери – різні!
- Також, чутливі до регістру опції команд, типу STATUS чи SHARED.
- Детальніше – див. документацію, `https://cmake.org/cmake/help/latest/manual/cmake-language.7.html`.

VERBOSE

- За замовчуванням CMake не повідомляє, як саме він компілює кожен із файлів:

```
$ make
Scanning dependencies of target hello1
[ 25%] Building CXX object CMakeFiles/hello1.dir/file1.cpp.obj
[ 50%] Building CXX object CMakeFiles/hello1.dir/file2.cpp.obj
[ 75%] Building CXX object CMakeFiles/hello1.dir/main_file.cpp.obj
[100%] Linking CXX executable hello1.exe
[100%] Built target hello1
```

- За потреби можна попросити його бути більш багатослівним.
- Один спосіб – ввімкнути детальний (багатослівний) вивід безумовно, в CMakeLists.txt:

```
SET(CMAKE_VERBOSE_MAKEFILE ON)
```

- Але такий вивід корисний лише для відладки, коли щось із компіляцією пішло не так. Тому, більш зручно вмикати його лише за потреби:

```
$ make VERBOSE=1
```

чи:

```
$ export VERBOSE=1
$ make
```

чи (використовуючи можливість створити змінну середовища лише для одного процесу):

```
$ VERBOSE=1 make
```

Вивід у VERBOSE режимі

```
D:/WorkComp/msys2_64_2015/mingw64/bin/c++.exe     -o CMakeFiles/hello1.dir/file1.cpp.obj -c D:/WorkT
eaching/Cpp/lectures/Projects/cmake_demo/file1.cpp
[ 50%] Building CXX object CMakeFiles/hello1.dir/file2.cpp.obj
D:/WorkComp/msys2_64_2015/mingw64/bin/c++.exe     -o CMakeFiles/hello1.dir/file2.cpp.obj -c D:/WorkT
eaching/Cpp/lectures/Projects/cmake_demo/file2.cpp
[ 75%] Building CXX object CMakeFiles/hello1.dir/main_file.cpp.obj
D:/WorkComp/msys2_64_2015/mingw64/bin/c++.exe     -o CMakeFiles/hello1.dir/main_file.cpp.obj -c D:/W
orkTeaching/Cpp/lectures/Projects/cmake_demo/main_file.cpp
[100%] Linking CXX executable hello1.exe
D:/WorkComp/msys2_64_2015/mingw64/bin/cmake.exe -E remove -f CMakeFiles/hello1.dir/objects.a
D:/WorkComp/msys2_64_2015/mingw64/bin/ar.exe cr CMakeFiles/hello1.dir/objects.a @CMakeFiles/hello1.d
ir/objects1.rsp
D:/WorkComp/msys2_64_2015/mingw64/bin/c++.exe    -Wl,--whole-archive CMakeFiles/hello1.dir/objects.a
 -Wl,--no-whole-archive  -o hello1.exe -Wl,--major-image-version,0,--minor-image-version,0 @CMakeFil
```

- Зверніть увагу на опції лінкера: `@CMakeFiles/hello1.dir/objects1.rsp` – це так званий response-file.
  - В загальному випадку, це спосіб передати довгий командний рядок на системах, де його довжина обмежена. Детальніше див. `https://gcc.gnu.org/wiki/Response_Files`.
  - В цьому випадку він містить список об'єктних файлів:
    CMakeFiles/hello1.dir/file1.cpp.obj
    CMakeFiles/hello1.dir/file2.cpp.obj
    CMakeFiles/hello1.dir/main_file.cpp.obj

Пошук зовнішніх бібліотек та інших пакетів

- Програма може залежати від сторонніх бібліотек.
- CMake вміє їх шукати та надавати відповідну інформацію для компіляції.
- Припустимо, нашій програмі потрібен Curses. Тоді, релевантні частини CMakeLists.txt можуть виглядати якось так:

```
find_package(curses)
.....................

ADD_EXECUTABLE(hello1  ${hello1_FILES})
TARGET_LINK_LIBRARIES(hello1 ${curses_LIBRARIES})
target_include_directories(OurLibrary PRIVATE
                                        ${curses_INCLUDE_DIRS})

# А ось так робити не варто -- застаріла практика!
# INCLUDE_DIRECTORIES(${curses_INCLUDE_DIRS})
```

Щодо INCLUDE_DIRECTORIES()

- **Увага**, популярний колись (а значить – поширений на stackoverflow) `INCLUDE_DIRECTORIES()` додає ці шляхи для всіх цілей побудови, тому є доволі грубим методом.
- Його використання не рекомендується[6] – більш коректним є метод per-target:

  ```
  target_include_directories(OurLibrary PRIVATE ..............)
  ```

- PRIVATE для `target_include_directories` означає, що ця залежність потрібна лише цілі, але не цілям, які від неї залежать.
- INTERFACE – лише тим, хто від неї залежать,
- PUBLIC – і тим і тим.

[6] Іноді і автор презентації грішить його використанням, але в більших проектах, над якими працює багато людей, це точно погана ідея!

Модулі CMake

- **Модуль CMake** – код, що описує способів пошуку пакету.
  - Його використовує виклик find_package().
- Із CMake йде кілька сотень таких модулів – для популярних бібліотек та пакетів.
- Модуль – просто програма мовою CMake, яка шукає певні файли, директорії, виконує виклик зовнішніх програм тощо, щоб знайти пакет, або переконатися – його немає.
  - Багато які модулі у своїй роботі покладаються на pkg-config, (`https://en.wikipedia.org/wiki/Pkg-config`) – отримавши доступ до великої бази конфігурацій бібліотек та інших пакетів.
- Можна створювати свої модулі для пошуку.

Стандартні змінні, оголошувані find_package для пакету XXX

- Кожен модуль, знайшовши свій пакет, оголошує змінні з інформацією про нього.
- Цей список може відрізнятися від пакету до пакету, але зазвичай включає, як мінімум це[7]:
  - `XXX_FOUND` – false, якщо пакет не знайдено.
  - `XXX_INCLUDE_DIRS` – список пошуку шляхів для файлів заголовків бібліотеки.
  - `XXX_LIBRARIES` – список бібліотечних файлів для лінкування. Включає їх абсолютні шляхи.
  - `XXX_DEFINITIONS` – оголошення, потрібні бібліотеці.
  - `XXX_EXECUTABLE` – ім'я виконавчого файлу пакету, (коли мова про інструменти, а не бібліотеки).
  - `XXX_LIBRARY_DIRS` – шляхи до бібліотек, зазвичай не потрібні, див. `XXX LIBRARIES`.
- `cmake --help-module FindCurses` – спосіб отримати подробиці конкретного модуля. Наприклад, згаданий модуль пошуку curses, створює `CURSES_HAVE_CURSES_H`, якщо є хедер curses.h, `CURSES_HAVE_NCURSES_H`, якщо ncurses.h.
  - Також, його роботою можна керувати, оголосивши деякі змінні перед викликом, наприклад: `CURSES_NEED_NCURSES` – потребуємо саме NCURSES.
- CMake доволі активно розвивається, тому багато прикладів в Інтернеті застаріли.
  - Наприклад, застарілим є вживання `XXX_INCLUDE_DIR` та `XXX_LIBRARY`

[7] Де XXX – назва модуля.

Пошук пакетів – 2

- Параметрами пакету можна, а часто й треба, керувати із командного рядка:

```
cmake -G"Unix Makefiles" -DCURSES_NEED_NCURSES=TRUE ..
```

- Зокрема, так можна підказувати модулю, як і де шукати його пакет:

```
cmake -DBOOST_ROOT=/usr/local/ \
    -DBoost_LIBRARY_DIRS=/usr/local/lib/boost_1.64
```

- Змінні, на які зважає модуль, описується в його документації.
- Інші команди сімейства пошуку: `FIND_PROGRAM`, `FIND_LIBRARY`, `FIND_FILE`, `FIND_PATH`.
- ◇ Зазвичай вони, використовуються при створенні модулів, а не безпосередньо в користувацьких CMakeLists.txt.
- Детальніше див. `https://cmake.org/cmake/help/git-master/manual/cmake-packages.7.html`.

Керування пошуком пакетів

- REQUIRED вказує генерувати помилку, якщо пакет не знайдено:

```
FIND_PACKAGE(SWIG REQUIRED)
```

  - Без цього робота CMake продовжиться, але SWIG_FOUND буде false.
- Шукати конкретну версію пакету та конкретних його частин:

```
set(Boost_USE_STATIC_LIBS        ON) # only find static libs
set(Boost_USE_MULTITHREADED      ON)
set(Boost_USE_STATIC_RUNTIME    OFF)
find_package(Boost 1.36.0
             COMPONENTS date_time filesystem system)
```

- Вище вказується мінімальна версія пакету. Якщо потрібна конкретна:

```
find_package(Boost 1.36.0 EXACT
             COMPONENTS date_time filesystem system)
```

- Якщо Ви хочете скористатися стороннім (скажімо, власного авторства), модулем – слід допомогти CMake знайти його:

```
set(CMAKE_MODULE_PATH ${CMAKE_MODULE_PATH}
          "${CMAKE_SOURCE_DIR}/cmake/Modules/")
```

- Див. також https://gitlab.kitware.com/cmake/community/wikis/doc/tutorials/How-To-Find-Libraries.

Умовні конструкції

- CMake дозволяє приймати рішення на основі того, що він виявив:

```
find_package (BZip2)
if (BZIP2_FOUND)
  include_directories(${BZIP_INCLUDE_DIRS})
  target_link_libraries (helloworld ${BZIP2_LIBRARIES})
endif (BZIP2_FOUND)
```

- Більш складний приклад:

```
find_package(Threads REQUIRED)
find_package(PCM)
find_package(PAPI)

if(PCM_FOUND)
    include_directories(${PCM_INCLUDE_DIRS})
    target_link_libraries (ConditionalQueue ${PCM_LIBRARIES})
    add_definitions(-DEXPERIMENTAL_USE_PCM)
elseif(PAPI_FOUND)
    include_directories(${PAPI_INCLUDE_DIRS})
    target_link_libraries (ConditionalQueue ${PAPI_LIBRARIES})
    add_definitions(-DEXPERIMENTAL_USE_PAPI)
endif()
target_link_libraries (ConditionalQueue ${CMAKE_THREAD_LIBS_INIT})
```

- ◇ Код з проекту для AKC, з певних причин тут використано include_directories, але пам'ятайте – це погана практика.

Аналіз системи

- Іноді потрібно таки зробити щось непереносиме. У такому випадку слід перевірити – а чи ми вчасно робимо це, чи платформа та, для якої потрібне таки нестандартне втручання.
  - Приклад – конфігурування шаблону студентських проектів:

```
if(MSVC)
  set(CMAKE_CXX_FLAGS "${CMAKE_CXX_FLAGS} /D_WIN32_WINNT=0x0600")
  if(CMAKE_CXX_FLAGS MATCHES "/W[0-4]")
    string(REGEX REPLACE "/W[0-4]" "/W4" CMAKE_CXX_FLAGS "${CMAKE_CXX_FLAGS}")
  else()
    set(CMAKE_CXX_FLAGS "${CMAKE_CXX_FLAGS} /W4 /wd4503")
  endif()
elseif(CMAKE_COMPILER_IS_GNUCC OR CMAKE_COMPILER_IS_GNUCXX)
  set(CMAKE_CXX_FLAGS "${CMAKE_CXX_FLAGS} -Wall -Wno-long-long -pedantic")
endif()

if(CMAKE_COMPILER_IS_GNUCC OR CMAKE_COMPILER_IS_GNUCXX)
  set(CMAKE_CXX_FLAGS "${CMAKE_CXX_FLAGS} -fno-omit-frame-pointer -g")
  IF(WIN32) # Should I use IF(MINGW) here?
    set(CMAKE_CXX_FLAGS "${CMAKE_CXX_FLAGS} -D_WIN32_WINNT=0x0600")
  ENDIF(WIN32)
  add_definitions(-D_GNU_SOURCE)
endif()
```

- **УВАГА!** Зміна `CMAKE_CXX_FLAGS` та аналогічних змінних CMake – грубе втручання! Воно припустиме лише за відсутності (простих) альтернатив. Категорично не варто так додавати шляхи до include, libs і т.д.!

- Приклад вище ілюструє, як можна підлаштовуватися під особливості системи – версію компілятора, версію ОС, тощо.
- Будьте, однак, обережні! Це – доволі грубе втручання в функціонування CMake! Воно категорично не рекомендоване до використання, якщо є більш високорівневі альтернативи. Тим не менше, перерахуємо відповідні змінні:
  - `CMAKE_CXX_FLAGS` – прапорці компіляції для компілятора C++,
  - `CMAKE_CC_FLAGS` – для компілятора C,
  - `LINK_FLAGS` – загальні опції лінкера,
  - `CMAKE_EXE_LINKER_FLAGS` – прапорці лінкера для генерації виконавчих файлів і т.д., і т.п.
- Якщо ви вже пішли на таке насильство над системою компіляції, часто краще додавати свої прапорці до тих, що вже є – CMake, в процесі роботи, поміщає туди всілякі цінні опції.
- За можливості, краще робити так (per-target):

```
set_target_properties(myexe_target PROPERTIES
                      COMPILE_FLAGS "-Wall")
```

Generator expressions

- Схожого результату можна досягнути, користуючись generator expressions (`https://cmake.org/cmake/help/latest/manual/cmake-generator-expressions.7.html`):

```
target_compile_definitions(myexe_target PRIVATE
   $<$<AND:$<COMPILE_LANGUAGE:CXX>, \
      $<CXX_COMPILER_ID:Clang>>:COMPILING_CXX_WITH_CLANG>
   $<$<AND:$<COMPILE_LANGUAGE:CXX>, \
      $<CXX_COMPILER_ID:Intel>>:COMPILING_CXX_WITH_INTEL>
  $<$<AND:$<COMPILE_LANGUAGE:C>, \
    $<C_COMPILER_ID:Clang>>:COMPILING_C_WITH_CLANG> )
```

- Детальніше їх поки не розглядатимемо.
- Однак, це дуже потужний інструмент, вважається майбутнім CMake.

Керування попередженнями

```
if(MSVC)
# Завжди W4:
  if(CMAKE_CXX_FLAGS MATCHES "/W[0-4]")
    string(REGEX REPLACE "/W[0-4]" "/W4" CMAKE_CXX_FLAGS "${CMAKE_CXX_FLAGS}")
  else()
    set(CMAKE_CXX_FLAGS "${CMAKE_CXX_FLAGS} /W4")
    # або й так -- Warnings as errors:
    # set(CMAKE_CXX_FLAGS "${CMAKE_CXX_FLAGS} /W4 /WX")
  endif()
elseif(CMAKE_COMPILER_IS_GNUCC OR CMAKE_COMPILER_IS_GNUCXX)
  # GCC і Clang враховують лише останню опцію, тому чистити
  # CMAKE_CXX_FLAGS не обов'язково.
  set(CMAKE_CXX_FLAGS "${CMAKE_CXX_FLAGS} -Wall -pedantic")
  # або й так:
  # set(CMAKE_CXX_FLAGS "${CMAKE_CXX_FLAGS} -Wall -Wextra -pedantic -Werror")
endif()
```

- CMake (поки – станом на 3.27, 2023) не підтримує керування Warning-ами, тому таке втручання може бути виправданим.
- Це ж, із використанням генераторів і більш локально, per-target – що є кращою практикою:

```
target_compile_options(${TARGET_NAME} PRIVATE
  $<$<CXX_COMPILER_ID:MSVC>:/W4 /WX>
  $<$<NOT:$<CXX_COMPILER_ID:MSVC>>:-Wall -Wextra -pedantic -Werror>
)
```

Умовні конструкції – 2

- Умовні команди: if(), elseif(), else(), endif().
- Логічні операції – NOT, AND, OR, наприклад: "`if(NOT WIN32)`" чи "`if(CMAKE_COMPILER_IS_GNUCC OR CMAKE_COMPILER_IS_GNUCXX)`".
- Конструкцій для перевірки різних спеціалізованих умов:
  - "`if(file1 IS_NEWER_THAN file2)`",
  - "`if(IS_ABSOLUTE path)`",
  - та багато інших – див. `https://cmake.org/cmake/help/latest/command/if.html`.
- Приклади:

```
IF(CMAKE_SYSTEM_NAME STREQUAL Linux) # Перевірка імені ОС

IF(DEFINED ENV{FOO_HOME}) # Чи існує змінна середовища
                          # -- зауважте, без $!
```

- Див. також `https://gitlab.kitware.com/cmake/community/wikis/doc/cmake/Examples`.

Надавайте перевагу керуванню властивостей окремих цілей

- Менш інтрузивні варіанти мають опцію PUBLIC чи PRIVATE, яка визначає, чи пропагувати цю опцію залежним цілям):

```
# Застарілий варіант:
# add_definitions(-D__STRICT_ANSI__)
# Краще:
target_compile_definitions(hello1 PUBLIC __STRICT_ANSI__)
# Синонім, все рівно підбере правильну опцію для компілятора:
target_compile_definitions(hello1 PUBLIC -D__STRICT_ANSI__)

# Але пам'ятаємо про кросплатформовість!
# MSVC та деякі інші компілятори такого ключа не знають:
target_compile_options(hello1 PRIVATE -fexceptions)

# Кожен виклик додаватиме відповідний -lname чи її аналог,
# зрозумілий цільовому компілятору:
target_link_libraries(hello1 PRIVATE xyz)

# Не дуже хороша практика:
include_directories(/path/to/includes)
# Краще:
target_include_directories(hello1 PRIVATE /path/to/includes)
```

Режими компіляції, з точки зору оптимізації

- CMake дозволяє згенерувати скрипти компіляції для компілювання із різними режимами оптимізації.
- Для цього потрібно передати йому відповідний CMAKE_BUILD_TYPE:

```
cmake <path> -G"<generator_name>" <other options...> \
             -CMAKE_BUILD_TYPE = Release
```

- Підтримуються наступні режими:
  - Release – максимальна оптимізація,
  - Debug – без оптимізації, з інформацією для відлагодження,
  - RelWithDebInfo – оптимізації, які не (дуже) заважають відлагодженню, плюс інформація для відлагодження,
  - MinSizeRel – оптимізація за розміром.

Мультиконфігурації

- В цілому, доводиться мати по окремій build-директорії для кожного режиму.
- Однак, деякі генератори, такі як Visual Studio чи "Ninja Multi-Config"[8] підтримують мульти-конфігурації.
- Скористатися ними можна так:

```
$ cmake .. -DCMAKE_CONFIGURATION_TYPES="Release;Debug" \
           -G"Ninja Multi-Config"
.........................
$ cmake --build . --config Debug
# Або:
$ cmake --build . --config Release
# Або більш безпосередньо:
$ ninja -f build-Debug.ninja
```

[8] Швидка альтернатива Make, яка, щоправда, орієнтується на автоматичну генерацію скриптів CMake чи іншими аналогічними засобами.

Емуляція мульти-конфігурації для Makefile

- Можна зробити щось таке:

```
ADD_CUSTOM_TARGET(debug
  COMMAND ${CMAKE_COMMAND} -DCMAKE_BUILD_TYPE=Debug
                                        ${CMAKE_SOURCE_DIR}
  COMMAND ${CMAKE_COMMAND} --build ${CMAKE_BINARY_DIR}
                                        --target all
  COMMENT "Switch CMAKE_BUILD_TYPE to Debug"
  )

ADD_CUSTOM_TARGET(release
  COMMAND ${CMAKE_COMMAND} -DCMAKE_BUILD_TYPE=Release
                                        ${CMAKE_SOURCE_DIR}
  COMMAND ${CMAKE_COMMAND} --build ${CMAKE_BINARY_DIR}
                                        --target all
  COMMENT "Switch CMAKE_BUILD_TYPE to Release"
  )
```

- Тоді, для компіляції у відповідному режимі:

```
make debug
# чи
make release
```

- Можна керувати опціями для конкретного режиму компіляції – Release, Debug, RelWithDebInfo, MinSizeRel:

  ```
  target_link_libraries(hello1 debug funcsd)
  ```

  - Бібліотека після "debug" буде використовуватися лише відлагоджувальної версії.
  - Можливі варіанти: debug, optimized, general, чутливе до регістру.
  - Див., також, документацію: `https://cmake.org/cmake/help/latest/command/target_link_libraries.html`.
- Іноді, для відлагодження, корисно подивитися всі властивості якоїсь цілі.
  - Приклад коду, який це робить: `https://stackoverflow.com/a/34292622` (Відповідь на запитання "How to print all the properties of a target in cmake?"). Покладається на get_property() та get_target_property(): `https://cmake.org/cmake/help/latest/command/get_property.html` і `https://cmake.org/cmake/help/latest/command/get_target_property.html`.
- Опція `--fresh` каже стерти стару конфігурацію і почати спочатку (3.24+).

CMake presets

- Базово, конкретна конфігурація CMake задається з командного рядка.
- Однак, з часом це стає все менш зручно – командні рядки зростають, над ними доводиться, на додачу до CMakeLists.txt, надбудовувати скрипти bash/Powershell – явно не кросплатформово чи довго конфігурувати в GUI IDE, що ще менше переноситься.
- В CMake 3.19 додали цікаву можливість – presets.
- Файл CMakeSettings.json дозволяє описати необхідні командні рядки та викликати їх багато лаконічніше.
  - Додатково, можна створити локальний файл із індивідуальними налаштуваннями: CMakeUserPresets.json.
  - В репозиторій проекту CMakeSettings.json комітиться, а CMakeUserPresets.json – ні.

CMake presets – приклад

- Нехай певний проект компілюється під MSVC якось так:

```
cmake ../04_count_words_1/ -G"Visual Studio 17 2022" \
 "-DCMAKE_TOOLCHAIN_FILE=<path>/vcpkg.cmake" \
  "-DCMAKE_GENERATOR_PLATFORM=x64" -CMAKE_BUILD_TYPE=Release
```

- Тоді, з CMakeSettings.json, наведеним далі, можна, замість цього командного рядка використати:

```
cmake .. --preset MSVC-x64-Release
```

- Подивитися список пресетів можна так:

```
$ cmake .. --list-presets
Available configure presets:

  "MSVC-x64"          - MSVC-x64
  "MSVC-x64-Debug"    - MSVC-x64-Debug
  "MSVC-x64-Release"  - MSVC-x64-Release
  ................
$ cmake .. --list-presets build  # Тощо
```

- Існують пресети для конфігурування, побудови (build), test, package та їх інтеграції – workflow.

```
{
    "version": 6,
    "cmakeMinimumRequired": {
        "major": 3,
        "minor": 25,
        "patch": 0
    },
    "configurePresets": [
        {
            "name": "MSVC-x64",
            "displayName": "MSVC-x64",
            "description": "Build for x86-64 using the MSVC and vcpkg toolchain file",
            "generator": "Visual Studio 17 2022",
            "architecture" : "x64",
            "toolchainFile": "c:/WorkComp/vcpkg2022/vcpkg/scripts/buildsystems/vcpkg.c
            "cacheVariables": {
                    "MY_CMAKE_VARIABLE": "Value"
            },
            "environment": {
                "MY_ENVIRONMENT_VARIABLE": "Test"
            }
        },
```

```
        {
            "name": "MSVC-x64-Debug",
            "inherits": "MSVC-x64",
            "displayName": "MSVC-x64-Debug",
            "description": "Debug Build for x86-64 using the MSVC and vcpkg toolchain
            "cacheVariables": {
                    "CMAKE_BUILD_TYPE": "Debug"
            }
        },
        {
            "name": "MSVC-x64-Release",
            "inherits": "MSVC-x64",
            "displayName": "MSVC-x64-Release",
            "description": "Release Build for x86-64 using the MSVC and vcpkg toolchai
            "cacheVariables": {
                    "CMAKE_BUILD_TYPE": "Release"
            }
        }
    ]
}
```

Інші presets

- build, workflow та інші пресети створюються аналогічно:

```
"buildPresets": [
    {
        "name": "MSVC-x64",
        "configurePreset": "MSVC-x64"
    },
    {
        "name": "MSVC-x64-Debug",
        "inherits": "MSVC-x64",
        "configurePreset": "MSVC-x64-Debug"
    }
],
"workflowPresets": [
{
    "name": "MSVC-x64",
    "steps": [
        {
          "type": "configure",
          "name": "MSVC-x64"
        },
        {
          "type": "build"
```

- Частини опису пресетів достатньо самодокументовані.
- Однак, для використання build, workflow та інші пресетів, потрібно, щоб і CMakeLists.txt і CCMakePresets.json перебували в поточній директорії, а шлях до out-of-source побудов задавався binaryDir в configurePresets.
- Може бути відносним:

```
"binaryDir": "${sourceDir}/build/default",
```

Presets – посилання

- `https://cmake.org/cmake/help/latest/manual/cmake-presets.7.html` – офіційна документація, з багатьма цікавими подробицями.
- `https://doc.qt.io/qtcreator/creator-build-settings-cmake-presets.html` – використання з Qt.
- `https://github.com/microsoft/vscode-cmake-tools/blob/main/docs/cmake-presets.md` – з VSCode.
- `https://www.jetbrains.com/help/clion/cmake-presets.html` – CLion.
- `https://blog.feabhas.com/2023/08/cmake-presets/` – огляд.
- `https://feabhasblog.wpengine.com/2023/08/cmake-presets/` – огляд на прикладі STM32 toolchain.

Інші опції – керування з командного рядка

- Шлях для інсталяції:

```
cmake .. -DCMAKE_INSTALL_PREFIX=/home/user/local \
         -DCMAKE_BUILD_TYPE=RelWithDebInfo
```

- Керування вибором компілятора:

```
CC=gcc-7 CXX=g++-7 cmake -G"Unix Makefiles" ..
```

  Варіант:

```
CC=gcc-7
CXX=g++-7
export CC CXX
cmake -G"Unix Makefiles" ..
```

- Див. також `cmake-gui`.

Створення бібліотек

- Зовсім коротко:

```
SET( mylib_SRCS library.cpp )
ADD_LIBRARY(my SHARED ${mylib_SRCS} )
# чи:
ADD_LIBRARY(my STATIC ${mylib_SRCS} )
```

- Можна навіть так – створити одночасно статичний та динамічний варіант:

```
add_library(bzlib STATIC ${SOURCES})
add_library(bzlib-shared SHARED ${SOURCES})
set_target_properties(bzlib-shared PROPERTIES OUTPUT_NAME
                                                    bzlib)
```

◇ Тут використано перейменування, щоб отримати класичні[9] імена файлів бібліотек: libbzlib.a, libbzlib.so.a, libbzlib.so – всі із одним базовим іменем та правильними розширеннями.

- Короткий огляд створення динамічних бібліотек: https://gernotklingler.com/blog/creating-using-shared-libraries-different-compilers-different-ope а детальніше вони розглядалися в іншій презентації.

[9] Підозрюю, згідно традиції autotools.

Динамічні бібліотеки для Windows – 1

- Експорт функцій із DLL під Windows потребує `__declspec(import)` та `__declspec(export)` або використання def-файлу[10].
- На жаль, коли компілювати MSVC, сам по собі код:
  `ADD_LIBRARY(my SHARED ${mylib_SRCS} )`
  створить динамічну бібліотеку (.dll), яка **нічого не експортуватиме**, і не створить бібліотеку імпорту (.lib).
  - ◇ Причина у різній поведінці за замовчуванням MSVC і GCC/Clang – перший не експортує нічого анотованого, другі експортують все, якщо немає анотацій[11].
- Про допомогу CMake у вирішенні цієї проблеми – див. наступний слайд.
- Деякі подробиці про створення Win DLL за допомогою CMake[12]:
  `https://blog.kitware.com/create-dlls-on-windows-without-declspec-using-new-cmake-export-a`
  див. також згадану вище статтю:
  `https://gernotklingler.com/blog/creating-using-shared-libraries-different-compilers-different-ope`

[10] Детальніше див. презентацію про бібліотеки.

[11] Більш строго, це різниця між підходами Windows та Unix, яку реалізовують їх компілятори.

[12] Застарілий текст на тему, якщо у вас CMake 2.8 або старіший:
`https://gitlab.kitware.com/cmake/community/wikis/doc/tutorials/BuildingWinDLL`.

Динамічні бібліотеки для Windows – 2

- Як вручну розставити в коді відповідні `__declspec` розглядається у презентації про бібліотеки.
- Якщо вручну розставляти помітки небажано[13], CMake може допомогти.
- Для цього є два варіанти:
  - Встановити символ `WINDOWS_EXPORT_ALL_SYMBOLS` в TRUE[14] в CMakeLists.txt:

    ```
    set(CMAKE_WINDOWS_EXPORT_ALL_SYMBOLS TRUE)
    ```

    чи за допомогою командного рядка:

    ```
    cmake -DCMAKE_WINDOWS_EXPORT_ALL_SYMBOLS=TRUE <.......>
    ```
  - При цьому, для глобальних даних, під час компіляції коду, який використовує бібліотеку, все рівно буде потрібно `__declspec(dllimport)`, як підказку лінкеру.
  - Скористатися `GENERATE_EXPORT_HEADER` – потребує змін до коду[15], лише допомагає створити макроси для анотування сутностей, що експортуються.

[13] Наприклад, портується код із Unix і це потребуватиме багато часу.

[14] Або 1, або ON

[15] Див. `https://stackoverflow.com/a/33297026` та `https://cmake.org/cmake/help/latest/module/GenerateExportHeader.html`.

Динамічні бібліотеки для Linux – керування RPATH

- Пошук динамічних бібліотек під час запуску виконавчих файлів в Linux керується rpath/LD_LIBRARY_PATH/runpath. Подробиці – у презентації про бібліотеки.
- Якщо в CMakeLists.txt є динамічні бібліотеки і виконавчі файли, що їх завантажують, CMake турбується, щоб "все працювало".
  - За замовчуванням, робиться це, встановлюючи runpath виконавчого файлу, щоб він містив абсолютний шлях до бібліотеки.
  - За замовчуванням, під час інсталяції, rpath/runpath очищається.
- Передбачено кілька змінні, які керують цим процесом:
  - `CMAKE_SKIP_RPATH` – за замовчуванням FALSE, не додавати RPATH до виконавчих файлів.
  - `CMAKE_SKIP_BUILD_RPATH` – FALSE, означає додавати повний RPATH до виконавчого файлу під час компіляції.
  - `CMAKE_INSTALL_RPATH` – "", RPATH після інсталяції.
  - `CMAKE_BUILD_WITH_INSTALL_RPATH` – FALSE, не використовувати INSTALL RPATH компілюючи (лише пізніше, інсталюючи).
  - `CMAKE_INSTALL_RPATH_USE_LINK_PATH` – FALSE, не додавати шляхи до бібліотек за межами директорії компіляції проекту до RPATH.
  - `CMAKE_BUILD_RPATH_USE_ORIGIN` – використовувати відносні шляхи в RPATH.
- Зауважте, під Linux (і Unix взагалі), виконавчі файли та бібліотеки можуть компілюватися під інсталяцію за конкретним шляхом та не будуть працювати належно, якщо їх зберегти за іншим шляхом.

- Опції з попереднього слайду діють на цілий проект. Можна також вказувати їх для конкретної цілі.
  - Для цього можна використати `set_target_propertie()` із `SKIP_BUILD_RPATH`, `BUILD_RPATH`, `BUILD_WITH_INSTALL_RPATH`, `INSTALL_RPATH`, `INSTALL_RPATH_USE_LINK_PATH`, `CMAKE_BUILD_RPATH_USE_ORIGIN` тощо[16].
  - Назви per-target опцій відрізняються відсутністю префікса `CMAKE_`.
- Приклад – встановлюємо лінкувати при інсталяції відносно виконавчого файлу:

```
set(CMAKE_INSTALL_RPATH "\$ORIGIN/../lib")
# Це ж, лише для виконавчого файлу -- не для бібліотеки:
set_target_properties(executable PROPERTIES INSTALL_RPATH
                                 "\$ORIGIN/../lib")
```

- Детальніше див. `https://gitlab.kitware.com/cmake/community/wikis/doc/cmake/RPATH-handling`, зокрема, там описано особливості MacOS.
- Також: `https://indico.cern.ch/event/848215/contributions/3591953/attachments/1923018/3181752/HSFPackagingRelocation.pdf` – "Helping C/C++ Packages be Relocatable".

[16] Повний список: `INSTALL_RPATH_USE_LINK_PATH`.

Залежності бібліотек

- Бібліотеки, як і інші цілі, можуть мати пов'язані із ними залежності: які бібліотеки потребує, де хедер її знаходяться, які опції компілятора використовувати і т.д.
- Їх можна поділити на дві частини – ті, що потрібні для компіляції бібліотеки і ті, що потрібні користувачам бібліотеки.
- Перші називають PRIVATE, другі – INTERFACE. Потрібні і там і там – PUBLIC.
- Наприклад, INTERFACE_COMPILE_FEATURES – включає засоби компілятора, потрібні користувачу даної бібліотеки, а COMPILE_FEATURES – бібліотеці, щоб скомпілюватися.

- Приклад:

```
target_include_directories(OurLibrary
  PUBLIC
    $<INSTALL_INTERFACE:include>
    $<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/include>
  PRIVATE
    ${CMAKE_CURRENT_SOURCE_DIR}/src
)

target_link_libraries(OurLibrary
  PUBLIC
    Boost::mpi Threads::Threads
  PRIVATE
    Boost::regex
)
```

- Тоді INCLUDE_DIRS міститиме src і include, а INTERFACE_INCLUDE_DIRS лише <..>/include.
- Аналогічно – Boost::regex потрібен лише всередині бібліотеки, а без Boost::mpi і Threads::Threads не обійдуться й її користувачі.

Імпорт бібліотек

- Трюк: додати бібліотеку, створену кимось іншим, яка не має модуля:

```
add_library(funcs SHARED IMPORTED)
if(WIN32)
  set_property(TARGET funcs PROPERTY IMPORTED_IMPLIB
    ${CMAKE_CURRENT_SOURCE_DIR}/../library/build_dir/libfuncs.dll)
else()
  set_property(TARGET funcs PROPERTY IMPORTED_LOCATION
    ${CMAKE_CURRENT_SOURCE_DIR}/../library/build_dir/libfuncs.so)
endif()

# Використання:
add_executable( program ${program_SOURCES}  )
target_include_directories(program PUBLIC ../library)
TARGET_LINK_LIBRARIES( program funcs )
```

- Зауважте: для Windows використано **IMPORTED_IMPLIB**, а решти Linux – **IMPORTED_LOCATION** – через це потрібен if(WIN32).
  - IMPORTED_IMPLIB може посилатися як на .dll, так і на .dll.a, якщо така бібліотека імпорту є.
- В цьому прикладі вважається, що бібліотека знаходиться за шляхом ../library відносно поточних souce-файлів, а компілювали її в ../library/build_dir.
- Якщо бібліотека присутня в системі і знаходиться в загальновідомих шляхах, скажімо, /usr/lib, такий підхід терпимий. Однак, в даній ситуації він **відверто неестетичний**, оскільки доводиться вказувати явний шлях до конкретної компіляції – build_dir. Більш правильне рішення – див. далі.

Імпорт бібліотек — Header only

- Header-only бібліотеки додаємо як INTERFACE:

```
add_library(simple_lib INTERFACE)
target_include_directories(simple_lib INTERFACE <...шлях...>)
target_link_libraries( simple_lib INTERFACE boost)
```

- Сама бібліотека з прикладу не лінкується, але, зауважте – потребує boost.
- Про це повідомляємо її користувачів викликом target_link_libraries(), INTERFACE вказує, що ця інформація потрібна користувачам бібліотеки, а не їй самій.

Імпорт програм

- Аналогічно, можна імпортувати не тільки бібліотеки, але й виконавчі файли.
- Приклад звідси: https://gitlab.kitware.com/cmake/community/wikis/doc/tutorials/Exporting-and-Importing-Targets

```
add_executable(generator IMPORTED)
set_property(TARGET generator PROPERTY IMPORTED_LOCATION \
                                  "/path/to/some_generator") # 2

set(GENERATED_SRC ${CMAKE_CURRENT_BINARY_DIR}/generated.c)
add_custom_command(
  OUTPUT ${GENERATED_SRC}
  COMMAND generator ${GENERATED_SRC}                          # 3
  )

add_executable(myexe src1.c src2.c ${GENERATED_SRC})
```

- Генератором може бути, наприклад, swig.
- Однак, і імпорт бібліотек і імпорт програм значно зручніший, якщо одночасно скористатися експортом.

Інсталяція та експорт – бібліотека

- Якщо бібліотека зробила щось таке (зверніть увагу на нову команду – install()):

```
add_library(funcs STATIC ${libfuncs_SOURCES})

# Звичайно, краще не пошук робити, а вказувати конкретний список!
file(GLOB funcs_HEADERS RELATIVE ${CMAKE_SOURCE_DIR} "*.h")
SET(funcs_INC_PATH include)
install(TARGETS funcs DESTINATION lib EXPORT funcs-targets)
install(FILES ${funcs_HEADERS} DESTINATION
        ${funcs_INC_PATH})
install(EXPORT funcs-targets DESTINATION lib/funcs)

target_include_directories(funcs PUBLIC
  "$<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}>"
  "$<INSTALL_INTERFACE:$<INSTALL_PREFIX>/${funcs_INC_PATH}>")
```

  То після інсталяції:

```
cmake -DCMAKE_INSTALL_PREFIX:PATH=<install-path> .. && make \
&& make install
```

  користувачі зможуть робити так, як показано на наступному слайді.

Інсталяція та експорт – користувачі бібліотеки

- Фрагмент CMakeLists.txt користувача бібліотеки:

```
include(${funcs_PREFIX}/lib/funcs/funcs-targets.cmake)

add_executable( program  ${program_SOURCES}  )
target_include_directories(program PUBLIC ${funcs_INC_DIRS} )
TARGET_LINK_LIBRARIES( program funcs )
```

  та компілювати:

```
cmake -Dfuncs_PREFIX:PATH=<funcs-install-path> \
      -G"Unix Makefiles" .. && make
```

- Детальніше – див. https://gitlab.kitware.com/cmake/community/wikis/doc/tutorials/Exporting-and-Importing-Targets.
- Щодо install() – див. https://gitlab.kitware.com/cmake/community/wikis/doc/cmake/Install-Commands.

Деякі важливі шляхи

- CMAKE_CURRENT_SOURCE_DIR містить шлях до директорії текстів програм, яка зараз опрацьовується,
- CMAKE_CURRENT_BINARY_DIR – шлях до поточної директорії побудови (build dir).
- Аналогічно, CMAKE_BINARY_DIR та CMAKE_SOURCE_DIR міститимуть відповідні кореневі директорії.
- ◇ Особливо важливо пам'ятати про відмінність між ними, коли працюєте із проектом, що містить піддиректорії із своїми CMakeLists.txt.
- Приклад використання – створення бібліотеки та виконавчого файлу разом:

```
SET(funcs_FILES file1.cpp file2.cpp
             file1.h file2.h)

add_library(funcs ${funcs_FILES})

SET(hello1_FILES main_file.cpp)

ADD_EXECUTABLE(hello1  ${hello1_FILES})

TARGET_LINK_LIBRARIES(hello1 funcs)
INCLUDE_DIRECTORIES(${CMAKE_CURRENT_SOURCE_DIR})
```

Сторонні пакети

- Спеціальний модуль, ExternalProject, підтримує можливість скачати, за потреби – пропатчити, скомпілювати, та, за потреби виконати тести чи заінсталювати сторонні пакети.
- Див. https://cmake.org/cmake/help/latest/module/ExternalProject.html.
- Приклад звідти:

```
include(ExternalProject)
ExternalProject_Add(foobar
  GIT_REPOSITORY    git@github.com:FooCo/FooBar.git
  GIT_TAG           origin/release/1.2.3
)

find_program(MAKE_EXE NAMES gmake nmake make)
ExternalProject_Add(secretsauce
 URL      http://intranet.somecompany.com/artifacts/sauce-2.7.tgz
          https://www.somecompany.com/downloads/sauce-2.7.zip
 URL_HASH MD5=d41d8cd98f00b204e9800998ecf8427e
 CONFIGURE_COMMAND ""
 BUILD_COMMAND     ${MAKE_EXE} sauce
)
```

- Симпатичний спосіб автоматично скачувати та компілювати Google test зразу в проект: `https://crascit.com/2015/07/25/cmake-gtest/`.
  - Як на мене – сильно спрощує застосування.
  - Відповідний репозиторій: `https://github.com/Crascit/DownloadProject`.
- Покладається на інший важливий засіб CMake – генерацію файлів конфігурації на льоту (типу славнозвісного config.h із autotools) – configure_file.

configure_file

- Генерація файлів на льоту – приклад із офіційної документації. Нехай є файл foo.h.in:

```
#cmakedefine FOO_ENABLE
#cmakedefine FOO_STRING "@FOO_STRING@"
```

- CMakeLists.txt може зробити так:

```
option(FOO_ENABLE "Enable Foo" ON)
if(FOO_ENABLE)
  set(FOO_STRING "foo")
endif()
configure_file(foo.h.in foo.h @ONLY)
```

- Тоді, якщо опція FOO_ENABLE ввімкнена, отримаємо в директорії компіляції такий foo.h:

```
#define FOO_ENABLE
#define FOO_STRING "foo"
```

configure_file

- Інакше:

```
/* #undef FOO_ENABLE */
/* #undef FOO_STRING */
```

- Не забуваємо про `include_directories(${CMAKE_CURRENT_BINARY_DIR})`.
- Детальніше див. `https://cmake.org/cmake/help/latest/command/configure_file.html`

Детальніше про перевірку системи

- Крім згадуваних вище конструкцій типу if(POSIX), CMake має й більш високорівневі інструменти.
- Модуль CheckIncludeFiles:

```
INCLUDE (CheckIncludeFiles)
CHECK_INCLUDE_FILES (malloc.h HAVE_MALLOC_H)
CHECK_INCLUDE_FILES (”sys/param.h;sys/mount.h” HAVE_SYS_MOUNT_H)
CONFIGURE_FILE(${CMAKE_CURRENT_SOURCE_DIR}/config.h.in
               \${CMAKE_CURRENT_BINARY_DIR}/config.h)
```

◇ Тоді, якщо config.h.in виглядає так:

```
#cmakedefine HAVE_MALLOC_H 1
#cmakedefine HAVE_SYS_MOUNT_H
```

◇ CMake виконає всі підстановки, створивши config.h якогось такого виду:

```
#define HAVE_MALLOC_H 1
#define HAVE_SYS_MOUNT_H
```

◇ Та дозволить скористатися умовною компіляцією:

```
#include ”config.h”

#ifdef HAVE_MALLOC_H
#include <malloc.h>
#else
#include <stdlib.h>
#endif
```

Інші вбудовані перевірки

- Існує ряд інших, аналогічних CheckIncludeFiles, модулів.
- Модуль CheckFunctionExists – перевірити наявність функції, спробувавши виконати лінкування (але див. `https://cmake.org/cmake/help/latest/module/CheckFunctionExists.html` – є нюанси!)
  `CHECK_FUNCTION_EXISTS(function variable)`
- CheckSymbolExists – перевіряє наявність символу, компілюючи, включивши вказані файли (розділені комою) та лінкуючи для функцій та змінних[17]. Див. `https://cmake.org/cmake/help/latest/module/CheckSymbolExists.html`.
  `check_symbol_exists(<symbol> <files> <variable>)`
- Для C++ краще скористатися CheckCXXSymbolExists.
- CheckLibraryExists – перевірка наявності бібліотеки та певної функції/функцій у ній.
- CheckTypeSize – перевірити існування та розмір типу.
- CheckPrototypeExists – перевірити наявність ф-ції в файлі заголовків.
- CheckCXXSourceCompiles та CheckCSourceCompiles – перевірити, чи переданий код компілюватиметься.
- Всі виклики встановлюють variable в TRUE, якщо знайшли.
- Детальніше див. `https://gitlab.kitware.com/cmake/community/wikis/doc/tutorials/How-To-Write-Platform-Checks`.

[17] Поведінка, більш типова для autotools, іноді корисна.

Написання модулів

- Тема ця велика, детально тут не будемо розглядати.
- Див. https://gitlab.kitware.com/cmake/community/wikis/doc/tutorials/How-To-Find-Libraries#writing-find-modules
- Простий приклад – самописний модуль FindPCM.cmake (для https://github.com/opcm/pcm):

```
# to be set before calling find_package:
#
#  PCM_PREFIX  Set this variable to the root installation of
#              libpcm if the module has problems finding the
#              proper installation path.
#
# Variables defined by this module:
#
#  PCM_FOUND          System has PCM libraries and headers
#  PCM_LIBRARIES      The PCM library
#  PCM_INCLUDE_DIRS   The location of PCM headers

find_path(PCM_PREFIX
    NAMES cpucounters.h
    HINTS ${PCM_PREFIX}/include ${PCM_PREFIX}
)
```

Продовження модуля FindPCM.cmake

```
find_library(PCM_LIBRARY
    NAMES libPCM.a PCM
    HINTS ${PCM_PREFIX}/lib ${PCM_PREFIX}/lib64 ${PCM_PREFIX}
)
set(PCM_LIBRARIES ${PCM_LIBRARY})

find_path(PCM_INCLUDE_DIRS
    NAMES cpucounters.h
    HINTS ${PCM_PREFIX}/include  ${PCM_PREFIX}
)
include(FindPackageHandleStandardArgs)

find_package_handle_standard_args(PCM DEFAULT_MSG
    PCM_LIBRARIES
    PCM_INCLUDE_DIRS
)

mark_as_advanced(
    PCM_PREFIX_DIRS
    PCM_LIBRARIES
    PCM_INCLUDE_DIRS
)
```

CPack

- Про CPack – інструмент підтримки інсталяції, теж поки не писатиму, лише приклад:

```
set(funcs_HEADERS file1.h file2.h)
set(funcs_FILES file1.cpp file2.cpp ${funcs_HEADERS})
add_library(funcs SHARED ${funcs_FILES})

set(funcs_INCLUDE_PATH include)
install(TARGETS funcs DESTINATION lib EXPORT funcs-targets)
install(FILES ${funcs_HEADERS} DESTINATION ${funcs_INCLUDE_PATH})
install(EXPORT funcs-targets DESTINATION lib/funcs)

target_include_directories(funcs PUBLIC \
  "$<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}>"
  "$<INSTALL_INTERFACE:$<INSTALL_PREFIX>/${funcs_INCLUDE_PATH}>")

set(CPACK_GENERATOR "DEB")
set(CPACK_DEBIAN_PACKAGE_MAINTAINER "Oleg Farenyuk")
INCLUDE(CPack)
```

- Створення пакету із бінарниками: make package
- Створення пакету із джерельними текстами (source): make package_source
- Детальніше див. `https://cmake.org/cmake/help/latest/module/CPack.html`.

Підпроекти

- Якщо в проект входять під-проекти, що знаходяться в своїх директоріях та мають свої CMakeLists.txt файли, зібрати все це до купи можна, зробивши так в головному файлі:

```
add_subdirectory( subproject_1 )
add_subdirectory( subproject_2 )
```

In-source builds

- Варто заборонити in-source компіляцію, особливо, якщо є субпроекти:

```
set(CMAKE_DISABLE_SOURCE_CHANGES ON)
set(CMAKE_DISABLE_IN_SOURCE_BUILD ON)
```

- Ці опції не документовані, і, ймовірно[18], ніколи не будуть документованими, але корисні.
- Приклад – головний CMakeLists.txt прикладів до курсу C++ виглядає так:

```
cmake_minimum_required(VERSION 3.1)

set(CMAKE_DISABLE_SOURCE_CHANGES ON) # Undocumented
set(CMAKE_DISABLE_IN_SOURCE_BUILD ON)

project(cpp_samples)

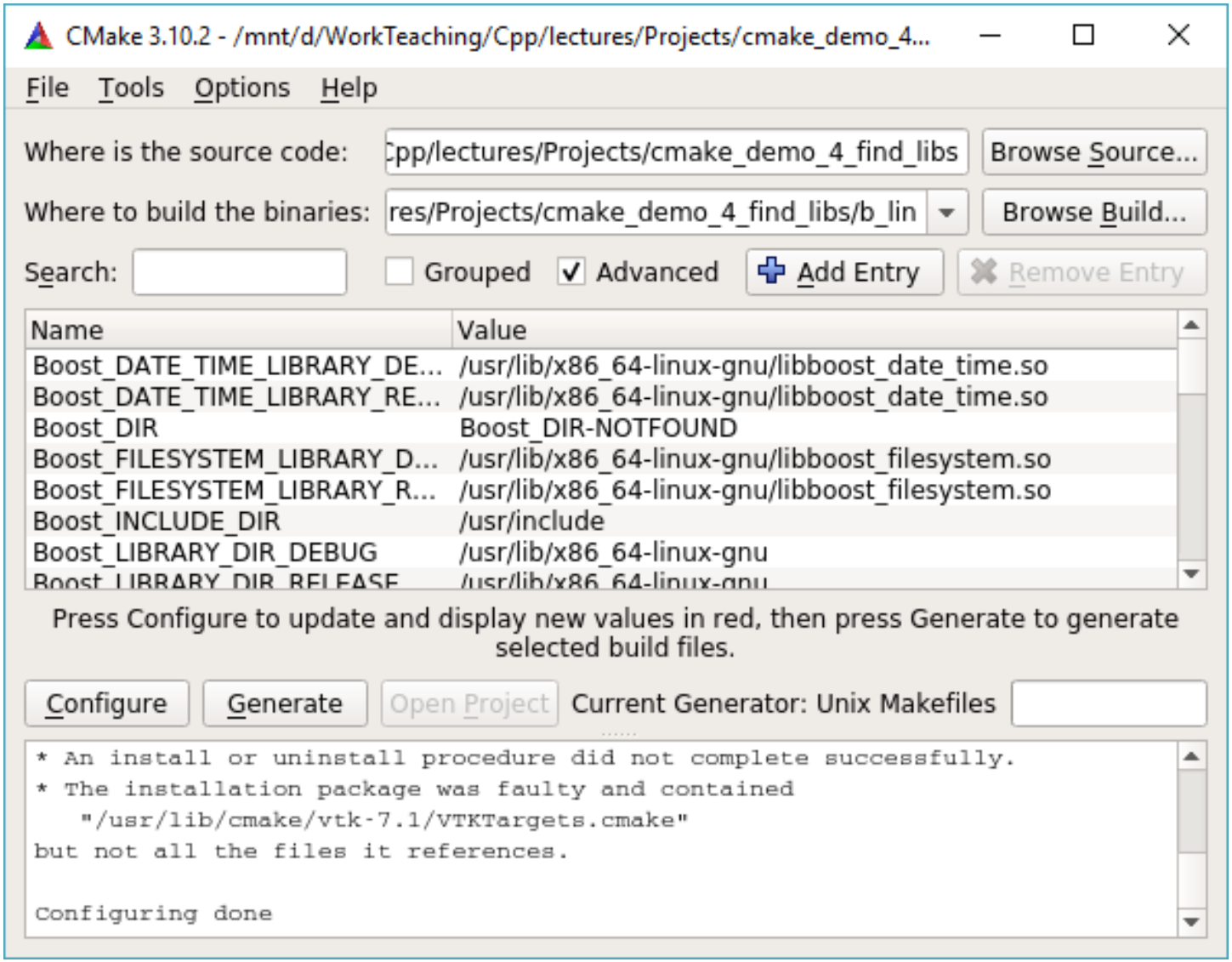
..............skipped.......................
add_subdirectory( l00_hello )
add_subdirectory( l01_cpp11_string_literals )
add_subdirectory( l01_var_sizes )
..............skipped.......................
```

Кожна із піддиректорій містить свій самодостатній CMakeLists.txt.

[18] Судячи з дискусій на форумах розробників.

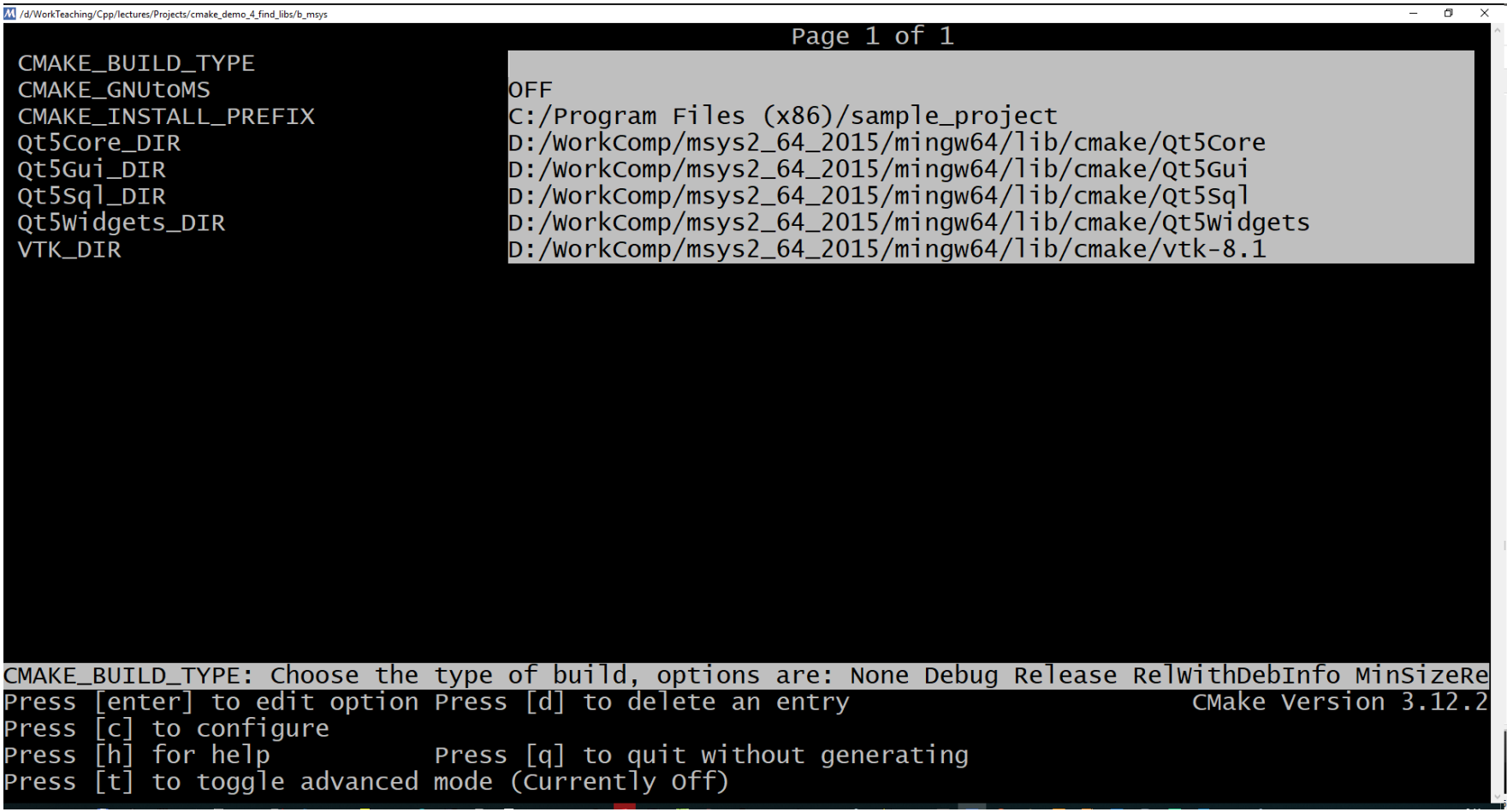
cmake-gui

- Корисний інструмент – рекомендую ознайомитися!



ccmake

- Існує також консольний інтерактивний інтерфейс – ccmake.



Кеш

- В директорії компіляції є файл CMakeCache.txt. Він містить збережені після попередніх конфігурувань значення.
- Хоча, на загал, керувати процесом конфігурування CMake краще із командного рядка, опціями -D чи -U – це дасть більш відтворюваний процес, іноді зручно спочатку розібратися, інтерактивно редагуючи CMakeCache.txt.
- Редагувати інші тимчасові файли – річ безглузда.

Цілі у скриптах компіляції, створені CMake

- Частину цілей ми вже зустрічали. Вважаючи, що працюємо із Makefile, це:

```
make
make clean
make install
make package
```

- Бувають і інші.
- Система побудови може, за потреби, викликати і CMake – якщо ви його змінили.
- Іноді вартує викликати CMake вручну. Наприклад, якщо ви додавали файли по масці і з часу конфігурування з'явилися нові чи зникли старі.

Керування інсталяцією

- Підтримується DESTDIR:

```
(1) make install DESTDIR="/some/absolute/path"
(2) make DESTDIR="/some/absolute/path" install
(3) DESTDIR="/some/absolute/path" make install
(4) export DESTDIR="/some/absolute/path
    make install
```

- make uninstall немає, але можна зробити так (у директорії компіляції):

```
xargs rm < install_manifest.txt
```

Власні генератори файлів

- Приклад, із змінами, з https://gitlab.kitware.com/cmake/community/wikis/FAQ

```
# Програма-генератор
ADD_EXECUTABLE(generate generate.c)
....
# Як нею користуватися:
ADD_CUSTOM_COMMAND(
  OUTPUT someoutput.c
  COMMAND generate ${CMAKE_CURRENT_SOURCE_DIR}/someinput.txt ${CMA
  DEPENDS generate ${CMAKE_CURRENT_SOURCE_DIR}/someinput.txt
  )

# Для чого вона -- хто від неї залежить:
ADD_CUSTOM_TARGET(driver ALL DEPENDS someoutput.c)
# Або:
ADD_EXECUTABLE(product product.c someoutput.c)
```

- ADD_CUSTOM_COMMAND може містити декілька команд, а поміж опцій може бути також: WORKING_DIRECTORY, COMMENT, VERBATIM

Інші команди

- Приклад із `https://gitlab.kitware.com/cmake/community/wikis/FAQ`

```
ADD_CUSTOM_COMMAND(
  TARGET Foo
  POST_BUILD
  COMMAND ${CMAKE_COMMAND} -E copy ${LIB_NAME}
                                   ${NEW_LIB_NAME}
)
```

- Ще одна важлива команда:

```
add_dependencies(test-foo foo)
```

гарантує, що спершу компілюватиметься foo, а вже потім – test-foo.

Дебагер CMake

- CMake потроху починає підтримувати відлагодження своїх скриптів.
  - За межами `message(STATUS "VAR=${VAR}")` та:

```
include(CMakePrintHelpers)
cmake_print_variables(MY_VARIABLE)

cmake_print_properties(
    TARGETS my_target
    PROPERTIES POSITION_INDEPENDENT_CODE
)
```

- По перше, це trace:

```
cmake -S . -B build --trace-source=CMakeLists.txt
```

  У варіанті з –trace-expand – друкуватиме розгортання змінних.

- По друге, хай експериментальна, але підтримка повноцінного виконання по рядках із інспектуванням змінних.
  - Детальніше про неї тут: `https://gitlab.kitware.com/cmake/cmake/-/issues/21510`.
  - Підтримується CLion[19]: `https://www.jetbrains.com/help/clion/cmake-debug.html`.
  - Підтримується MS Visual Studio: `https://devblogs.microsoft.com/cppblog/cmake-debugger-allows-you-to-debug-your-cmake-scripts-and-more/`.
  - ~~Можна очікувати в майбутньому~~ підтримки й іншими інструментами.

[19] На базі наробіток `https://sysprogs.com/w/introducing-cmake-script-debugger/`.

Реєстр Windows

- Підтримується доступ до реєстру Windows:

```
GET_FILENAME_COMPONENT(SDK_ROOT_PATH
  "[HKEY_LOCAL_MACHINE\\SOFTWARE\\PACKAGE;Install_Dir]"
                                        ABSOLUTE CACHE)

FIND_FILE(BOOT_DOT_INI boot.ini PATHS
                      [HKEY_CURRENT_USER\\Environment;HOMEDRIVE])
```

(Приклади звідси: https://gitlab.kitware.com/cmake/community/wikis/FAQ).

Підключення багатопоточності

- Багатопоточність[20] на різних платформах і субплатформах (MinGW/Cygwin/MSYS, наприклад) підключається трішки по різному.
- CMake вміє про це потурбуватися[21].

```
find_package (Threads REQUIRED)

<defining target>
target_link_libraries(target_name Threads::Threads)
```

- Попросити надавати перевагу pthread можна за допомогою `CMAKE_THREAD_PREFER_PTHREAD`. Якщо потрібен прапорець `-pthread`, то ще й `THREADS_PREFER_PTHREAD_FLAG`:

```
set(CMAKE_THREAD_PREFER_PTHREAD TRUE)
set(THREADS_PREFER_PTHREAD_FLAG TRUE)
find_package(Threads REQUIRED)
```

- Див. також `https://cmake.org/cmake/help/latest/module/FindThreads.html`.
- **Не використовуйте** ручної передачі прапорців в CMAKE_C_FLAGS чи інших застарілих практик, типу використання CMAKE_THREAD_LIBS_INIT!

[20] Ключова тема курсу АКС.

[21] Увага, тут описано сучасний (на 2018-2023) підхід. В Інтернеті є багато застарілих прикладів – обережно із ними!

Boost

- Підключаємо:

```
find_package(Boost 1.36.0 REQUIRED COMPONENTS date_time
                                  filesystem system)

SET(simple_ls_FILES simple_ls.cpp)
ADD_EXECUTABLE(simple_ls ${simple_ls_FILES})
include_directories(${Boost_INCLUDE_DIRS})
target_link_libraries(simple_ls ${Boost_LIBRARIES})
```

- Керування особливостями:

```
set(Boost_USE_STATIC_LIBS        ON) # only find static libs
set(Boost_USE_MULTITHREADED      ON)
set(Boost_USE_STATIC_RUNTIME    OFF)
find_package(Boost 1.36.0
             COMPONENTS date_time filesystem system)
```

Вказується мінімальна версія пакету. Якщо потрібна конкретна:

```
find_package(Boost 1.36.0 EXACT
             COMPONENTS date_time filesystem system)
```

- Опцій і варіантів у буста безліч, так що див. https://cmake.org/cmake/help/latest/module/FindBoost.html!

Boost-ремарка

- **Увага!** Станом на весну 2020 це вже було зайвим і навіть шкідливим. Використовуйте, лише якщо зустрінетеся із такою проблемою.
- Під MSYS2 із Boost 1.68 та GCC 8.2 чогось вилазили загадкові помилки лінкування, явно пов'язані із libboost_system. Пофіксилося так:

```
if(WIN32) # Strange errors...
target_compile_definitions(simple_ls PRIVATE
   BOOST_SYSTEM_NO_DEPRECATED=1
   BOOST_ERROR_CODE_HEADER_ONLY=1)
endif()
```

- Станом на літо 2019 проблема ніби зникла – в кілька ітерацій. В майбутньому цей слайд може бути видалено.

VTK

- VTK, як "рідний" проект CMake, підключається особливо легко:

```
find_package(VTK REQUIRED)
include(${VTK_USE_FILE})
```

OpenCV

```
FIND_PACKAGE(OpenCV REQUIRED )
INCLUDE_DIRECTORIES( ${OPENCV_INCLUDE_DIR} )

ADD_EXECUTABLE(Scalar Scalar.cxx)
TARGET_LINK_LIBRARIES(Scalar ${OpenCV_LIBS})
```

SWIG

- З офіційної документації:
  http://www.swig.org/Doc3.0/SWIGDocumentation.html

```
# This is a CMake example for Python

FIND_PACKAGE(SWIG REQUIRED)
INCLUDE(${SWIG_USE_FILE})

FIND_PACKAGE(PythonLibs)
INCLUDE_DIRECTORIES(${PYTHON_INCLUDE_PATH})

INCLUDE_DIRECTORIES(${CMAKE_CURRENT_SOURCE_DIR})

SET(CMAKE_SWIG_FLAGS "")

SET_SOURCE_FILES_PROPERTIES(example.i PROPERTIES
                                      CPLUSPLUS ON)
SET_SOURCE_FILES_PROPERTIES(example.i PROPERTIES
                              SWIG_FLAGS "-includeall")
SWIG_ADD_MODULE(example python example.i example.cxx)
SWIG_LINK_LIBRARIES(example ${PYTHON_LIBRARIES})
```

- Пакет підтримки санітайзерів із CMake: `https://github.com/arsenm/sanitizers-cmake`.
- Використання виглядає так:

```
find_package(Sanitizers)

sanitizer_add_blacklist_file("blacklist.txt")

add_executable(some_exe foo.c bar.c)
add_sanitizers(some_exe)
```

- Підтримує SANITIZE_ADDRESS, SANITIZE_MEMORY, SANITIZE_THREAD or SANITIZE_UNDEFINED.
- Детальніше – див. за посиланням.

Бонус: VCPKG+MSVC2017+

- Потроху і MSVC обростає адекватними менеджерами пакетів. vcpkg дозволяє скачати та скомпілювати обрані пакети автоматично. Детальніше див. `https://docs.microsoft.com/en-us/cpp/vcpkg?view=vs-2017`.
- Припустимо, проект містить boost і vtk. В директорії побудови, із Developer Command Prompt for VS2017[22]:

```
some_path> git clone  https://github.com/Microsoft/vcpkg
some_path> cd vcpkg
some_path\vcpkg> bootstrap-vcpkg.bat
```

- Якщо спомпілювалося успішно, продовжуємо[23]:

```
some_path\vcpkg> ./vcpkg install boost
some_path\vcpkg> ./vcpkg install vtk
some_path\vcpkg> cd <project_build_dir>
project_build_dir> cmake .. -DCMAKE_TOOLCHAIN_FILE=\
      <some_path>\vcpkg\scripts\buildsystems\vcpkg.cmake
```

Процес займе доволі багато часу – вказані пакети компілюватимуться! Це мінус, але і плюс – сумісність із ~~поточним компілятором~~ гарантована.

[22] В припущенні, що ми користуємося VS2017 – нагадую, різні C++ компілятори бінарно між собою не сумісні, а версії VS, наприклад,VS2017 і VS2015 вважаються різними, хоча VS2015+ підтримують сумісність.

[23] Про всяк випадок – з-під MSYS воно не дуже працює, хоча потім користуватися з нього можна нормально.

- Послідовність дій, безперечно, не унікальна! Наведена для прикладу.
- Щоб інтегрувати цю компіляцію бібліотек в MSVS IDE, для всіх проектів її користувача, виконайте: `vcpkg integrate install`, (за першого використання потребуватиме права адміністратора).
- `vcpkg search` дозволяє переглянути список пакетів, `vcpkg export` – створити zip-архів чи NuGet дистрибутив для розповсюдження.
- Оновлення версії бібліотек виглядає так: `vcpkg upgrade --no-dry-run`. За замовчуванням, без відповідного ключа, робить лише репетицію. Можливо, в цьому є сенс.
- Копій vspkg в систем може бути багато – із різними бібліотеками, їх версіями, налаштуваннями.
- Нагадую також про опцію для CMake:

```
"-DCMAKE_TOOLCHAIN_FILE=\
    d:/WorkComp/vcpkg/scripts/buildsystems/vcpkg.cmake"
```

- В інших ОС чи середовищах розробки подібні засоби є давно. Нарешті і в VS підключення C, і особливо – C++, бібліотек стає менш проблемним.

- Підтримка Boost.locale із ICU вимагає певних "танців з бубном"[24]:

```
vcpkg install libzip:x64-windows libarchive:x64-windows icu:x64-windows
vcpkg install boost-locale[icu]:x64-windows
# Фейлиться, але важливо спробувати -- щоб закешувало.
# Тоді: <шляхДо>\vcpkg\buildtrees\boost-locale\src\
#                 ost-1.69.0-429423b19e\build\Jamfile.v2
# замінити відповідний рядок на:
# ICU_PATH =  "<шляхДо>/vcpkg/installed/x64-windows/" ;
# а lib-файли ICU для debug
# (з <шляхДо>/vcpkg/installed/x64-windows/debug/lib/icu*d.lib)
# скопіювати в <шляхДо>/vcpkg/installed/x64-windows/lib/
# інакше воно на їх відсутність скаржиться.
# Мова про вирішення цієї проблеми:
# https://stackoverflow.com/questions/19751189/boostlocaleto-lower-throw-bad-
cast-exception
vcpkg install boost:x64-windows
# Слід знести існуючий boost-mpi і вручну заінсталювати той,
# що воно скачало. (Або із elevated cmd працювати -- що гірша ідея)
vcpkg install boost-mpi:x64-windows
# Тепер не потребувало додаткового шаманства.
vcpkg install boost-regex[icu]:x64-windows
```

- Якщо ICU не підключилося, Boost.locale все рівно працюватиме – із обмеженим функціоналом, зокрема, boost::locale::to_lower() кидатиме виключення bad_cast.
- **Станом на 2023, ніби вже працює "з коробки".**

[24] Станом на весну-літо 2019 – такі речі ефемерні, хоча досвід вирішувати відповідні проблеми – вічний.

- Інсталяція інших, важливих на АКС, бібліотек:

```
vcpkg install folly[*]:x64-windows
vcpkg install tbb:x64-windows gsl:x64-windows
vcpkg install gtest:x64-windows glog:x64-windows
```

- VTK трохи капризувало:

```
# Перед цим слід додати до MSVC, його засобами,
# -- інсталятором, ATL/MFC.
vcpkg install vtk:x64-windows # failed
# Закоментував рядки, пов'язані із ccache в
# <шляхДо>\vcpkg\buildtrees\jsoncpp\src\1.8.4-15e6f27349\
#                                          CMakeLists.txt
vcpkg install vtk:x64-windows # Повторити -- тепер OK
```

Оновлення пакетів

- ```
  git stash
  git pull
  git stash pop
  .\bootstrap-vcpkg.bat
  # Якщо було оновлення, далі може капризувати, типу
  # пропонувати знести якісь пакети: ”Error: cannot locate new
  #  portfile for boost:x64-windows. Please explicitly remove
  # this package with ‘vcpkg remove boost:x64-windows‘”.
  .\vcpkg upgrade
  .\vcpkg upgrade  --no-dry-run
  # А ось нижче не мало б бути потрібним, однак -- довелося.
  vcpkg install libarchive:x64-windows # Десь відвалилася...
  vcpkg install tbb:x64-windows        # Десь відвалилася...
  ```
  ```

Використання із CMake/CLion

- CMake слід підказати так званий Toolchain file, створений VCPKG для вашого MSVC (конкретної версії):

  ```
  -DCMAKE_TOOLCHAIN_FILE=D:\WorkComp\vcpkg\scripts\buildsystems\vcpk
  ```

- Для CLion варто ще додати:

  ```
  -DCMAKE_EXPORT_COMPILE_COMMANDS=ON
  ```

- `https://cmake.org/documentation/`, "CMake Documentation" – офіційна документація. Як на мене – абсолютно жахлива з точки usability.
- `https://cmake.org/cmake/help/latest/guide/tutorial/index.html` – **офіційний підручник, хороший**.
- `https://cgold.readthedocs.io/en/latest/`, "CGold: The Hitchhiker's Guide to the CMake" – **непогана відправна точка 1**.
- `https://gitlab.kitware.com/cmake/community/wikis/home`, "Офіційна wiki" – багато на які питання там є відповідь. УВАГА! – частина сторінок застарілі, звертайте увагу на дату.
- Ken Martin, Bill Hoffman, "Mastering CMake" – офіційний підручник, виданий на папері. В принципі, знаходиться в Інеті.
- `https://github.com/TheErk/CMake-tutorial`, "CMake tutorial presentation and code" – ще один хороший підручник.
- `https://mirkokiefer.com/cmake-by-example-f95eb47d45b1` – "CMake by Example" і `https://codingnest.com/basic-cmake-part-2/` – "Basic CMake, part 2: libraries", пара зовсім простеньких інструкцій.
- `https://pabloariasal.github.io/2018/02/19/its-time-to-do-cmake-right/`, "It's Time To Do CMake Right", огляд сучасного способу використання. Репозиторій із прикладами: `https://github.com/pabloariasal/modern-cmake-sample`.
- Effective Modern CMake – gist з правилами.
- "An Introduction to Modern CMake" – гарний сучасний підручник.

Нюанси

- https://stackoverflow.com/questions/36648375/what-are-the-differences-between-imported-target-and-interface-li – різниця між IMPORTED і INTERFACE.
- http://mesos.apache.org/documentation/latest/cmake-examples/ – про те ж, що і попередня, але ґрунтовніше.
- https://stackoverflow.com/questions/47416090/cmake-is-there-a-difference-between-set-propertytarget-and-set-ta – про різницю між set_property() і set_target_properties().
- https://gitlab.kitware.com/cmake/community/wikis/doc/cmake/Install-Commands
- https://gitlab.kitware.com/cmake/community/wikis/doc/cmake/CrossCompiling – важлива, але мало описана тема.
- https://gitlab.kitware.com/cmake/community/wikis/doc/tutorials/How-To-Write-Platform-Checks
- https://gitlab.kitware.com/cmake/community/wikis/doc/cmake/languages/Assembler – зовсім слабеньке, але яке є...

- `https://cmake.org/cmake/help/latest/manual/cmake-buildsystem.7.html` – загальний огляд із офіційної документації.
- `https://cmake.org/cmake/help/latest/manual/cmake-compile-features.7.html` – діагностика можливостей компілятора та: `https://cmake.org/cmake/help/latest/variable/CMAKE_LANG_COMPILER_ID.html` – ідентифікатори компіляторів.
- `https://cmake.org/cmake/help/latest/command/target_include_directories.html`.
- `https://cmake.org/cmake/help/latest/manual/cmake-packages.7.html` – пошук і створення пакетів.
- `https://gitlab.kitware.com/cmake/community/wikis/doc/tutorials/Exporting-and-Importing-Targets`.
- "Modern On-Target Embedded System Testing with CMake and CTest" – дещо нетипове використання.

Альтернативний підручник – на прикладі Embedded задач

- https://blog.feabhas.com/2021/07/cmake-part-1-the-dark-arts/
- https://blog.feabhas.com/2021/07/cmake-part-2-release-and-debug-builds/
- https://blog.feabhas.com/2021/08/cmake-part-3-source-file-organisation/
- https://blog.feabhas.com/2021/09/cmake-part-4-windows-10-host/

